Mémento Python

Ce document ne prétend ni à l'exhaustivité ni à une parfaite consistance. Il ne se substitue pas aux documentations officielles Python, scipy/numpy . Il s'agit seulement d'un aide-mémoire facilitant l'utilisation de Python comme outil pour la pratique des mathématiques. Le mode interactif de Python permet si nécessaire d'avoir des informations plus précises : type(*objet*) # donne de type de l'*objet*
dir(*objet_ou_fonction*), help(*objet_ou_fonction*), *objet_ou_fonction* ? # affiche les méthodes et attributs pouvant concerner l'*objet ou la fonction*.

Calculs dans $\mathbb{Z}$, $\mathbb{R}$, $\mathbb{Q}$, $\mathbb{C}$: ***types integer, float, Rational, complex***.

	import numpy as np
5//3 # quotient de la division euclidienne de 5 par 3 5%3 # reste de la division euclidienne de 5 par 3 int(-1.9) # troncature entière int(*booléen*) #fct caractéristique True → 1 et False → 0 round(1/6,4) # float arrondi du réel 1/6 à 10^{-4} près 1.2**3.4 # $(1,2)^{3,4}$ =$e^{3,4\ln(1,2)}$ car 1,2>0	np.pi # float approximation de π np.sin, np.cos, np.tan # fcts pouvant s'appliquer à une liste np.arcin, np.arccos, np.arctan np.sqrt, np.log, np.exp # peuvent s'appliquer à un complexe

2+3j # nombre complexe 2+3 *i*
1j # nombre complexe *i*
(2+3j).real # partie réelle
(2+3j).imag # partie imaginaire
(2+3j).conjugate() # conjugué
abs(2+3j) # module
(1+1j)**(1/3) # donne UNE racine cubique de 1+*i*
	# idem mais pouvant s'appliquer à une liste
	np.real()
	np.imag()
	np.conj()
	np.absolute()
	np.angle(2+3j) # approximation d'un argument de 2+3 *i*

Les chaînes de caractères et le calcul symbolique

str(123)# conversion de l'integer 123 en chaîne de caractères '123' 'ABC'+'123' # concaténation de deux chaînes de caractères 3*'ABC' # concaténation donnant 'ABCABCABC' ord('A') # code ascii du caractère A chr(65) # chaîne constituée du caractère dont le code ascii est 65 *chaîne*.split() # liste des mots (séparés par des espaces dans *chaîne*) *chaîne*.join(*liste_de_chaînes*) # concaténation de toutes les chaînes de caractères *chaîne*.upper(), *chaîne*.lower() # conversion d'un texte en majuscule ou minuscules *chaîne*.replace(*old*,*new*) # remplace les caractères dans une chaîne eval(*chaîne*) # interprète la chaîne de caractères comme du code Python

Tableaux à une dimension : liste ou, pour les calculs dans $\mathbb{R}^n$, $\mathbb{C}^n$ (voire $\mathbb{R}^{\mathbb{N}}$ et $\mathbb{C}^{\mathbb{N}}$) : array et Matrix

	import numpy as np # le type array permet de manipuler des tableaux de nombres dont le cardinal est fixé lors de sa création

list(...) # conversion en liste
list(range(*fin_exclue*)) # entiers POSITIFS OU NULS
list(range(*début* : *fin_exclue*)) # liste d'entiers consécutifs
list(range(*début* : *fin_exclue* : *pas*) # liste d'entiers
[f(i) for i in *liste*] # création de liste en compréhension
[f(i) for i in *liste* if g(i)] # g(i) est un booléen
[1,2]+[3,4] # concaténation
3*[1,2] # concaténation de 3 fois la même liste
	np.array([1,2,3]) # matrice ligne
	np.array([[1],[2],[3]]) # matrice colonne
	np.zeros(3),np.ones(3)# matrice ligne de trois 0 ou trois 1
	np.linspace(*début,fin_incluse,nombre d'éléments*)
	np.arange(*début,fin_incluse,pas*)
	np.array([1,2])+np.array([3,4]) # somme dans $M_{1,2}(\mathbb{R})$
	3*np.array([1,2]) # multiplication externe dans $M_{1,2}(\mathbb{R})$

len(L) # nombre d'éléments de L max(L), min(L) # minimum et maximum de L sum(L) # somme des éléments de L	np.array([3,4])**2 # array des carrés des termes 1/np.array([3,4]) # array des inverses des termes np.linalg.norm(...) # norme eucl. canonique

élément in L # booléen égal à True ssi *élément* est dans L
map(*liste*,*fonction*) # liste des images des éléments de *liste* par *fonction*
set(L) # ensemble des valeurs de L (sans doublon)	

L[0] # premier terme de la liste L	idem pour array
L[i] # terme d'index i la liste L L[-1] # dernier terme de la liste L L[*début* : *fin(exclue)* : *pas*] # sous-liste selon les index L[*début*:],L[:*fin(exclue)*],L[:],L[::*pas*] L[i],L[j]=L[j],L[i] #permutation de deux éléments	
L.count(*élément*) #nombre d'occurrence d'*élément* dans L L.sort() # modifie L en triant ses éléments L.pop() #enlève le dernier terme L et renvoie sa valeur L.pop(i) #enlève le terme d'index i de L et renvoie sa valeur L.append(*élément*) # ajoute l'*élément* à la fin de L L.insert(i,*élément*) # insert *élément* à l'index i dans L et décale les autres L.index(*élément*) # index de la première occurrence de *élément* dans la liste L	np.vdot([1,2],[3,4]) #produit scalaire canonique np.cross([1,2,3],[4,5,6]) # produit vectoriel np.transpose([1,2]) #matrice colonne

Tableaux à 2 dimensions : liste de listes ou, pour des calculs dans $M_{n,\,p}(\mathbb{R})$ ou $M_{n,\,p}(\mathbb{C})$, array et Matrix :
Une matrice $(a_{i,\,j}) \in M_{n,\,p}(K)$ est définie par la liste de ses n lignes (chacune des lignes étant une liste de p nombres) : les index *ligne_i* et *colonne_j* commencent à 0.

	import numpy as np
M=[[a(i+1,j+1) for j in range(p)] for i in range(n)]#liste des lignes len(M) # nombre de lignes len(M[0]) # nombre de colonnes	np.array([[1,2,3],[4,5,6]]) # 2 lignes, 3 colonnes np.zeros((*nb_ de_lignes*,*nb_de_colonnes*)) # couple np.ones((*nb_ de_lignes*,*nb_de_colonnes*)) # couple np.eye(*taille*) # matrice identité np.diag([1,2,3]) # matrice diagonale
M[*ligne_i*] # renvoie la liste indexée par *ligne_i* M[*ligne_i*][*colonne_j*] # terme (M[*ligne_i*])[*colonne _j*] [[M[i][*colonne_j*]] for i in range(n)]#liste des lignes de *colonne_j*	M[*ligne_i*,*colonne_j*] # terme $a_{i+1;\,j+1}$ M[*ligne_i*] # array 1D M[*ligne_i*:*ligne_i*+1, :] # array 2D M[:,*colonne_j*] # array 1D M[:,*colonne_j*:*colonne_j*+1] # array 2D
# les opérations dans $M_{n,\,p}(K)$ ne sont pas obtenues par manipulations simples sur les listes de listes.	3*M+N # combinaisons linéaires de matrices M*N # produit des termes 2 à 2 ≠ M×N np.dot(M,N) # produit matriciel M×N M.dot(N) # produit matriciel M×N M**3 # cube de chaque terme ≠ M^3 np.linalg.matrix_power(M,3) # M^3 si M est carrée np.linalg.inv(M) # M^{-1} si M est inversible
	np.transpose(M),np.trace(M), np.linalg.det(M), np.linalg.norm(M) np.poly(M) #polynôme caractéristique cf Polynomial np.linalg.eigvals(M) # spectre dans $\mathbb{C}$ np.linalg.eig(M) # couple : spectre, matrice de passage

Calculs dans $\mathbb{R}[X]$ ou $\mathbb{C}[X]$ ***: évaluation, coefficients, racines, dérivation, intégration, division euclidienne***

```
from numpy.polynomial import Polynomial
```

```
P=Polynomial([-3,2,0,1]) # P est le polynôme X^3+2 X−3
P(1+2j) # évaluation de P (1+2 i)
```

```
P.coef # renvoie array([-3,2,0,1])
P.degree()
P.roots() # racines complexes
```

```
P.deriv() # polynôme dérivé
P.deriv(2) # dérivée seconde du polynôme P
P.integ()# primitive de P s'annulant en 0
Q=P.integ(n,[ y_{n-1} ,..., y_0 ]) # Q^(n)=P , Q(0 )= y_0 ,...., Q^(n-1) (0)= y_{n-1}
```

```
Q=P//D # quotient dans la division euclidienne du polynôme P par le polynôme D
R=P%D # R est le reste dans la division euclidienne du polynôme P par le polynôme D
```

Fonctions scalaires d'une variable réelle : représentation graphique, équation numérique, dérivation, intégration développements limités, séries entières, séries de Fourier

```
def f(x) :
    …
    return ...
```

```
X=... # liste ou array des valeurs prises par x
Y=[f(x) for x in X] # liste des images
import matplotlib.pyplot as plt
plt.plot(X,Y,color='r',linestyle=':',marker='o')# plot(liste_abscisses,liste_ordonnées)
plt.axis('equal') # repère orthonomal
plt.axis(x_min,x_max,y_min,y_max) # bornes du repère
plt.show() #affichage de la courbe
```

```
from scipy.otpimize import fsolve
fsolve(f, x_0 ) # approximation d'une solution de f (x )=0 par itérations avec valeur
initiale x_0 . risque de ne pas converger ou de converger vers une autre racine que
celle la plus proche de x_0 . Dans le doute une dichotomie est préférable.
```

```
[(Y[i+1]-Y[i])/(X[i+1]-X[i]) for i in range(len(X)-1)] # liste des taux d'accroissements
```

```
from scipy.integrate import quad
quad(f,0,1) # couple (valeur approchée de ∫_0^1 f (t )dt , majoration de l'erreur)
quad(f,0,np.inf) # couple (valeur approchée de ∫_0^{+∞} f (t )dt , majoration de l'erreur)
```

```
[sum[ a_n *x**n for n in range(10)] for x in X] # somme partielle ∑_{n=0} a_n x^n
import numpy as np
[ a_0 +sum[ a_n *np.cos(n* ω *x)+ b_n *np.sin(n* ω *x)] for n in range(1,10)] for x in X]
# somme partielle de la série de Fourier
```

Fonctions vectorielles d'une variable réelle : courbe paramétrée(2D ou 3D), longueur d'une courbe paramétrée ; vecteur vitesse, point stationnaires, développements limités

```
def f(t) :
    …
```

```
    return (…,...) # couple pour la 2D, triplet pour la 3D
```

```
T=... # liste des valeurs prises par t
X=[f(t)[0] for t in T] # la première composante de f(t)        est une abscisse
Y=[f(t)[1] for t in T] # la seconde composante de f(t)         est une ordonnée
import matplotlib.pyplot as plt
plt.plot(X,Y) # ligne brisée passant par les points (x(t);y(t))
plt.show()
```

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
ax=Axes3D(plt.figure())
ax.plot(X,Y,Z) # listes des abscisses, des ordonnées et des cotes
plt.show()
```

```
sum([np.sqrt((X[i+1]-X[i])**2+(Y[i+1]-Y[i])**2) for i in range(len(T)-1)] #
approximation de la longueur de la courbe paramétrée par sommation de longueur de
segments
```

Fonctions scalaires de plusieurs variables : courbe d'équation f (x ; y)=0 , surface d'équation z= f (x ; y) , extremum, dérivées partielles

```
def f(x,y) :
    …
    return ... # float
import numpy as np
X = … # liste ou array des valeurs de x
```

```
Y = … # liste ou array des valeurs de y
Z = [[f(x,y) for x in X] for y in Y]
import matplotlib.pyplot as plt
```

```
plt.contour(X,Y,Z,0) # courbe d'équation f(x , y )=0
plt.contour(X,Y,Z,[ z1 , z2 , z3 ]) # courbes d'équations f ( x , y )=z1 etc...

plt.show()
```

```
min([min(Z[i]) for i in range(len(Z))]) # recherche du minimum
X, Y = np.meshgrid(X, Y) # création du maillage
ax=Axes3D(plt.figure())
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.plot_surface(X,Y,Z) # maillage abscisses × ordonnées et cotes
plt.show()
```

Fonctions vectorielles de plusieurs variables : système

```
def f(v) : # v étant une liste ou un array , v[0], v[1] et v[2] correspondent à x, y et z
    …
    return … , ... # tuple de float
```

```
from scipy.optimize import root
sol=root(f,[1,2,3]) # méthode itérative initialisée au point (1,2,3)
sol.success # booléen pour la convergence de la méthode
sol.x # dernier point lors des itérations
```

Équations différentielles scalaires du premier ordre, système d'équations différentielles, application aux équations différentielles scalaires linéaire homogène d'ordre n

```
from scipy.integrate import odeint
X=... # liste ou array décrivant l'intervalle de résolution
def f(y,x) :
    ...
    return … # pour résoudre y'(x)= f ( y , x )
Ly=odeint(f, y0 ,X) # approximation des y(X[i]) pour la solution vérifiant la condition
```

initiale y(X[0])= y_0

```
def f(Y,x) :
    return np.array([... ,...]) # système différentiel
```

$$\begin{pmatrix} Y'[0] \\ \vdots \\ Y'[n-1] \end{pmatrix} = \begin{pmatrix} f(Y,x)[0] \\ \vdots \\ f(Y,x)[n-1] \end{pmatrix}$$

LY=odeint(f,np.array[y_0 ,... , y_{n-1}],X) # approximation des arrays Y(X[i]) pour la solution vérifiant la condition initiale Y(X[0])=(y_0 ,... , y_{n-1})

```
def f(Y,t) : # ici Y=[ y , y' ,..., y^(n-1) ] et on veut résoudre Y'=AY
    n=... #odre de l'équation différentielle linéaire
    A=np.zeros([n,n])
   for i in range(n-1):
       A[i,i+1]=1
    A[n-1,:]=[ a_0 ,..., a_{n-1} ] # pour résoudre y^(n) =a_0 y +...+a_{n-1} y^(n-1)
    return A.np.dot(Y)
```

Ly=odeint(f,np.array[y_0 ,... , y_{n-1}],X)[0] # première composante de Y, approximation des y (X[i]) pour la solution vérifiant les conditions initiales $y^{(k)}$(X[0])= y_k

Simulation d'expériences aléatoires : une variable aléatoire X, couple de variables aléatoires (X,Y)

```
import numpy.random as rd
rd.randint(1,7) # variable aléatoire suivant une loi uniforme sur  1; 6  (entiers)
rd.random # variable aléatoire suivant une loi uniforme sur l'intervalle  [0;1[
rd.binomial(1,0.3)  # variable aléatoire suivant une loi de Bernoulli de paramètre p=0,3
rd.binomial(10,0.3)  # variable aléatoire suivant une loi binomiale  n=10 et p=0,3
rd.geometric(0.5) # variable aléatoire suivant une loi géométrique de paramètre p=0,3
rd.poisson(4) # variable aléatoire suivant une loi de Poisson de paramètre λ=4
```

```
L=[]
for k in range(n) :
    x=...
    y=... # peut dépendre de x
    L.op([x,y]) # création d'une liste de n couples empiriques  (x;y)
```